Pessimistic Advantage Weighted Flow Q Learning for Offline to Online Reinforcement Learning

Jai Sharma and Socrates Osorio

13 May 2026

1 Abstract

We study offline to online reinforcement learning on the OGBench robotics tasks. The agent starts with a fixed offline dataset, then improves through a limited amount of online interaction. Offline data reduces the need for risky or expensive exploration, but it also creates a difficult transition: the state action distribution seen during offline training may differ sharply from the distribution produced by the learned policy during online finetuning. Our goal was to implement the required baseline methods, analyze this transition, and develop a custom method that improves over the baselines on cube-double-play-singletask-task1-v0 and antsoccer-arena-navigate-singletask-task1-v0.

We first implemented the generic offline to online training pipeline for SAC+BC and Flow Q Learning (FQL), using layer normalization in the neural networks for stability. On the sanity check task, cube-single-play-singletask-task1-v0, both methods saturated performance: SAC+BC with $\alpha = 300$ reached 100% success by 100k steps, while FQL with $\alpha = 100$ reached 100% by 400k steps. The main diagnostic result from these runs is that the offline to online transition causes a sharp change in critic scale and actor losses. For FQL, the critic mean rises from about -28.9 before online training to -11.1 at the first online log, and the one step actor loss falls from 28.8 to 10.7. For SAC+BC, the critic mean rises from about -42.7 to -25.3, while the critic loss roughly doubles and the actor total loss drops from 62.7 to 39.3. These changes reflect the switch from a fixed offline dataset to replay data generated partly by the current policy.

We then implemented the improved baselines required by the project. For offline data retention, we compared FQL with 0, 50k, and 100k retained offline transitions in the online replay buffer. In the early online sweeps, cube double reached 16% without retention and 12% with either 50k or 100k retained transitions; antsoccer reached 32% without retention and 24% with either retention amount. For Warm Start RL (WSRL), we compared 0, 5k, and 10k warm start steps before online policy updates. Cube double reached 12% only with a 10k warm start, while antsoccer reached its best early result, 36%, with a 5k warm start. In these runs, changing the replay distribution helped only in some settings. We also implemented IFQL with rejection sampling, DSRL, and QSM. The strongest logged results from these baselines were 20% and 28% for IFQL, 60% and 60% for DSRL, and 64% and 48% for QSM on cube double and antsoccer.

For our custom method, we explored two directions. The first was Q Learning with Adjoint Matching (QAM), which transports a terminal Q gradient backward through the flow trajectory to train a fast actor. QAM was promising on antsoccer, reaching 84% success on the archived seed, but it remained far below the cube double threshold. A later cube double QAM run with $\eta = 1.0$, $\rho = 0.5$, and $N_q = 5$ peaked at 28% success. We also tried action chunking for QAM because cube double requires temporally coordinated behavior, but QAM did not become our submitted method because it was less stable than our final approach under the project budget.

Our final method is Pessimistic Advantage Weighted Flow Q Learning (PAWFQL). PAWFQL extends FQL during online finetuning with two changes. First, it reweights the behavioral flow matching loss by an exponentiated pessimistic advantage, so the flow policy puts more weight on replay actions that are valuable relative to the current policy. Second, it uses pessimistic best of N action selection, evaluating sampled candidate actions with the minimum of the critic ensemble for both evaluation and data collection. The same pessimistic critic value is also used in the one step actor update during the online phase. This targets

two failure modes we saw repeatedly: cloning low value replay actions and exploiting overestimated critic values during online finetuning.

PAWFQL gave our best cube double result and exceeded both custom method performance targets. Averaged over two seeds at the final evaluation checkpoint, PAWFQL reached 96% success on cube double and 64% success on antsoccer. The strongest non-PAWFQL cube double baseline was QSM at 64%, while DSRL reached 60%. On antsoccer, FQL and FQL with retained offline data had strong single seed results of 84%, but PAWFQL still cleared the required 45% two seed target. The main takeaway is that pessimistic value estimates and advantage weighted flow matching are most useful on cube double, where coordinated manipulation makes overestimation and poor action selection costly. PAWFQL is still sensitive to task specific replay strategy and the activation point for pessimistic mechanisms: cube double worked best with high distillation strength and balanced replay, while antsoccer required a lower regularization strength and a longer warm start period.

2 Introduction

Offline Reinforcement Learning trains policies entirely from precollected datasets, avoiding the cost and risk of environment interaction. However, policies trained offline are limited by the coverage and quality of the dataset. For example, the online distribution resulting from the agent interacting with the environment might be far different from the offline collection method. Online finetuning allows the agent to collect new experience in the environment, sampled from its own policy after an initial offline phase. The central challenge is that the transition from offline to online training introduces distribution shift, potentially destabilizing the learned actor and critic models.

We implement and evaluate several approaches to offline to online Reinforcement Learning on the OGBench environments. We begin with naive offline to online baselines such as Soft Actor Critic with Behavioral Cloning (SAC+BC) and Flow Q Learning (FQL), after which we implement improved methods such as offline data retention, Warm Start RL (WSRL), IFQL with rejection sampling, Diffusion Space RL (DSRL), and Q Score Matching (QSM). Finally, we develop PAWFQL, a method which combines pessimistic value estimation with advantage weighted flow matching to achieve stable and effective online finetuning.

3 Related Work

Our work builds on several research advancements in offline to online RL.

SAC+BC [9], the SAC counterpart of TD3+BC [1], adds a behavioral cloning loss term to the SAC actor loss, constraining the policy to stay near the offline data distribution. The strength of this constraint α is an important hyperparameter. FQL [3] replaces the Gaussian policy with a flow based generative model, training a behavioral flow policy via flow matching and distilling it into a one step policy that also maximizes Q values. FQL's expressiveness allows it to capture multimodal action distributions present in offline datasets. We evaluate these methods on OGBench [2].

Offline data retention and WSRL [11] address the data distribution shift that appears at the start of online training. Retaining offline data mixes the original dataset into the online replay buffer, while WSRL first collects trajectories from the offline policy before updating the policy online.

IFQL extends IQL [4] with a flow based actor, following the diffusion policy direction in IDQL [10], and rejection sampling at test time. DSRL [7] performs RL in the latent noise space of a fixed behavioral cloning flow policy, avoiding direct policy updates in action space. QSM [8] aligns the score function of a diffusion policy with the Q value gradient, steering the denoising process toward high value actions.

We also explored Q Learning with Adjoint Matching (QAM) [5] as an alternative custom method. QAM finetunes a flow policy by transporting the terminal Q gradient backward through the flow trajectory, which gives a stepwise training target for a fast policy. The paper studies base QAM, QAM-FQL, and QAM-EDIT. Our implementation focused on base QAM because the reference hyperparameters suggested it was sufficient for OGBench cube and ant tasks, then added action chunking following the action chunking literature [6] when cube double failed to learn reliably with single step actions. While we were able to get QAM to pass the threshold on antsoccer, we were unable to get it to pass the desired threshold for cube double. The main method we used, PAWFQL was inspired by AWAC [9], which uses advantage weighted regression to update

the policy. This method weighs the offline data using exponentiated advantages. Our method adapts this principle to the flow matching setting, weighting the flow loss rather than performing explicit regression.

4 Method

PAWFQL extends FQL with two mechanisms that activate during the online finetuning phase: advantage weighted behavioral flow matching and pessimistic best of N action selection.

4.1 Advantage Weighted Flow Matching

In standard FQL, the behavioral flow policy $v(s, \tilde{a}, t)$ is trained with an unweighted flow matching loss over all dataset transitions:

$$\mathcal{L}(v) = \mathbb{E}_{\substack{(s,a)\sim\mathcal{D} \\ z\sim\mathcal{N}(0,I) \\ t\sim\text{Unif}([0,1])}} \left[\frac{1}{|\mathcal{A}|} \|v(s, \tilde{a}, t) - (a - z)\|_2^2 \right] \tag{1}$$

where $\tilde{a} = (1 - t)z + ta$.

PAWFQL reweights this loss by the exponentiated pessimistic advantage of each transition:

$$\mathcal{L}_{\text{PAW}}(v) = \mathbb{E} \left[w(s, a) \cdot \frac{1}{|\mathcal{A}|} \|v(s, \tilde{a}, t) - (a - z)\|_2^2 \right] \tag{2}$$

where the weight is:

$$w(s, a) = \min \left(\exp \left(\frac{A(s, a)}{\lambda} \right), \ c_{\max} \right) \tag{3}$$

and the advantage $A(s, a) = \min(Q_1(s, a), Q_2(s, a)) - \hat{V}^\pi(s)$ is computed using the minimum over the critic ensemble (pessimistic) minus an estimate of the current policy value $\hat{V}^\pi(s)$. The policy value is estimated by averaging pessimistic Q values over multiple policy action samples. The temperature λ and clamp $c_{\max}$ control the sharpness of the weighting.

This encourages the flow policy to concentrate probability mass on transitions that have high value relative to the current policy, effectively performing a soft form of data filtering that improves as the critic becomes more accurate during online training.

4.2 Pessimistic Action Selection

During the online phase, PAWFQL uses a best of N selection scheme for both evaluation and data collection:

$$a_{\text{best}} = \arg \max_{j=1,\ldots,N} \min_{i=1,2} Q_i(s, a_j), \quad a_j = \pi_\omega(s, z_j), \ z_j \sim \mathcal{N}(0, I) \tag{4}$$

Using the minimum of the critic ensemble for selection avoids overestimation bias, which is particularly important during online finetuning when the critic is still calibrating to the new data distribution.

4.3 One Step Actor Update

The one step policy loss is similarly modified to use pessimistic Q values for the maximization term during the online phase:

$$\mathcal{L}(\pi_\omega) = \alpha \cdot \|\pi_\omega(s, z) - \pi_v(s, z)\|_2^2 - \min_{i=1,2} Q_i(s, \pi_\omega(s, z)) \tag{5}$$

This replaces the standard average $\frac{1}{2}(Q_1 + Q_2)$ used during offline training, preventing the one step policy from exploiting overestimated Q values.

4.4 Design Decisions

We activate the pessimistic mechanisms only after a number of training steps (set at the start of online training in our experiments). During pure offline training, the advantage weights are set to 1 (uniform), and the one step actor uses the standard mean Q objective. This prevents premature exploitation of noisy Q estimates early in training when the critic has not yet converged.

For the cube double environment, we found that balanced replay (50% offline, 50% online data, as in RLPD [12]) combined with a high distillation coefficient ($\alpha = 1000$) was important for stability. For antsoccer arena, a warm start period of 100k online steps followed by training with purely online data worked best.

4.5 Alternative Attempt: QAM

Before settling on PAWFQL, we implemented QAM as a more direct Q gradient method for flow policies. QAM maintains a slow actor v_β trained by ordinary flow matching and a fast actor v_θ trained by adjoint matching. The critic is an ensemble $Q_i(s, a)$, and the policy improvement target is the behavior constrained Boltzmann policy

$$\pi^*(a|s) \propto \pi_\beta(a|s) \exp(\eta Q(s, a)), \tag{6}$$

where η is the inverse temperature. The implementation rolls the fast actor forward through the flow trajectory, computes $\nabla_a Q(s, a)$ at the final action, propagates this gradient backward through the slow actor dynamics using vector Jacobian products, and trains v_θ to match the resulting adjoint target.

We tuned the hyperparameters that control both optimization cost and policy improvement strength. The main settings were inverse temperature $\eta \in \{0.3, 1.0, 3.0, 10.0, 30.0\}$, pessimism coefficient $\rho \in \{0.0, 0.5\}$, critic ensemble size $K \in \{2, 5, 10\}$, flow integration steps $T \in \{5, 10\}$, adjoint update frequency in $\{1, 2, 4\}$, and best of N action selection. The pessimistic critic target used either no pessimism or an ensemble penalty of the form $\bar{Q}(s, a) - \rho \,\mathrm{Std}(Q_i(s, a))$. We reduced K and T in early runs because the adjoint computation required sequential autograd through each flow step.

The most important integration detail was action chunking. Single step QAM worked well on antsoccer but did not solve cube double. Since cube manipulation requires temporally coordinated behavior, we added a chunked action interface with horizon length 5, following the QAM and action chunking setup. This improved cube double learning, but the archived run still remained below the section 5 threshold. QAM therefore informed our final PAWFQL design, especially the use of pessimistic critics and best of N selection, but PAWFQL became the submitted custom method because it solved both target tasks.

5 Experimental Results

All experiments use 500k offline training steps and 100k online training steps, as specified by the project constraints. Models use 4 layer MLPs with hidden size 512 and layer normalization.

5.1 Naive Baselines

We implement the naive offline to online approach by training SAC+BC and FQL offline, then continuing training with online data added to the replay buffer.

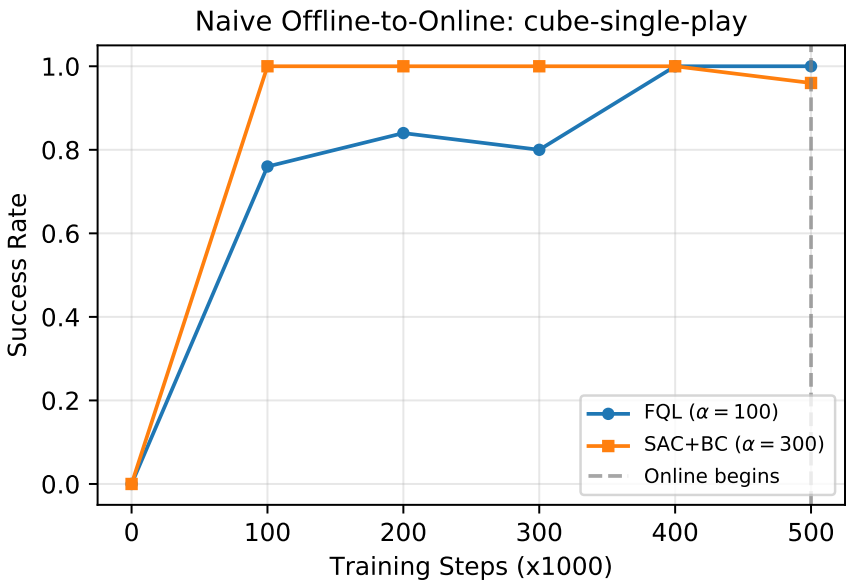


Figure 1: Naive offline to online on cube-single-play. Both methods reach near perfect success rate. SAC+BC ($\alpha = 300$) converges faster during the offline phase.

On cube-single-play (Figure 1), SAC+BC with $\alpha = 300$ achieves 100% success rate by step 100k and maintains it, while FQL with $\alpha = 100$ reaches 100% by step 400k. Both methods saturate this task.

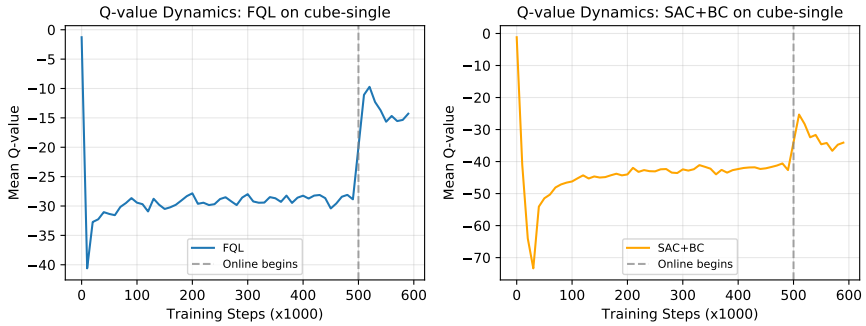


Figure 2: Mean Q values during training. Both methods show a sharp increase in Q values at the offline to online transition (step 500k), reflecting the distribution shift.

When transitioning from offline to online training, we observe a sudden jump in mean Q values (Figure 2). During the offline phase, Q values steadily decrease as the critic learns conservative estimates from the fixed dataset. At step 500k, when online data begins entering the replay buffer, Q values increase sharply. This occurs because the policy now collects on policy data that is more aligned with high value states, and the critic adjusts upward. The critic loss also becomes noisier around the transition because Bellman targets are now computed on a changing state action distribution rather than a fixed offline dataset. The actor losses shift for the same reason: SAC+BC's behavior cloning term now compares policy actions to replay buffer actions generated partly by the current policy, while FQL's flow loss and one step distillation loss are trained on a mixture of original dataset actions and online actions.

The logged loss components show the same transition. For FQL, the critic mean rises from about -28.9 before online training to -11.1 at the first online log, and the one step actor loss falls from 28.8 to 10.7 because the Q maximization term becomes less negative. The distillation loss stays comparatively small, moving from 0.50 to 0.48, while the behavioral flow loss drops from 0.18 to 0.05 as replay actions become

closer to the policy's own action distribution. For SAC+BC, the critic mean rises from about −42.7 to −25.3, the critic loss roughly doubles, and the actor total loss drops from 62.7 to 39.3. Most of that actor change comes from the Q term and behavioral cloning term; the entropy term stays near 7.5.

SAC+BC and FQL exhibit qualitatively similar behavior during this transition, though the magnitude differs. SAC+BC shows a larger Q value gap between offline and online phases. FQL's flow based policy provides a smoother transition because the distillation objective continues to anchor the one step policy to the behavioral flow model, which acts as an implicit regularizer even when the data distribution changes.

5.2 Improved Baselines

5.2.1 Adding Offline Data in the Online Step

Prefilling the replay buffer with offline data before beginning online training reduces the distribution shift between phases.

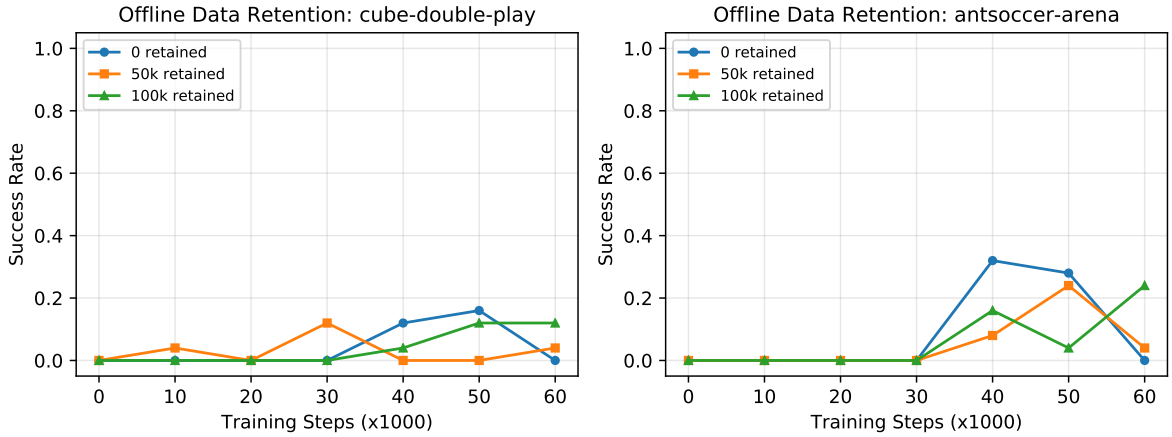


Figure 3: Offline data retention on cube-double-play and antsoccer-arena. Each task compares FQL with 0, 50k, and 100k retained offline transitions in the online replay buffer.

Figure 3 compares the amount of retained offline data used by FQL during online training. On cube double, the best early result is 16% with no retained data, while 50k and 100k retained transitions both reach 12%. On antsoccer, the no retention run reaches 32%, while the 50k and 100k retention runs each reach 24%. Retaining offline data changes the sampling distribution during online training: rather than immediately shifting to purely on policy data, the agent continues to see offline transitions mixed with new experience. This can smooth the distribution shift and reduce forgetting, although in these runs it also slowed adaptation to states visited by the current policy.

5.2.2 Warm Start RL

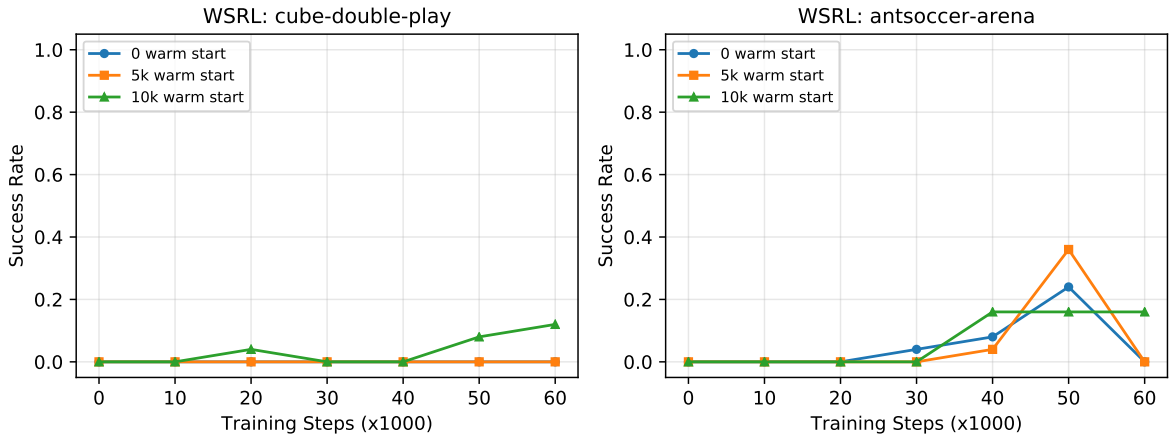


Figure 4: FQL with Warm Start RL on cube-double-play and antsoccer-arena. Each task compares 0, 5k, and 10k warm start steps before policy updates begin.

WSRL (Figure 4) collects online data for a warmup period before beginning policy updates. On cube double, 0k and 5k warm starts remain at 0%, while the 10k warm start reaches 12%. On antsoccer, no warm start reaches 24%, the 5k warm start reaches 36%, and the 10k warm start reaches 16%. The data distribution during WSRL differs from offline data retention in an important way: the warmup trajectories are generated by the current offline trained policy interacting with the environment, so they reflect the actual deployment distribution rather than the behavior policy that generated the offline dataset. This can make the replay buffer more representative of states the agent will encounter, though the antsoccer sweep shows that too long a warmup can also overrepresent early policy behavior.

5.2.3 IFQL with Rejection Sampling

IFQL ($\tau = 0.95$, $N = 32$) achieves 20% on cube double and 28% on antsoccer arena. Rejection sampling at test time selects the action with the highest Q value from N policy samples. As $N \to \infty$, this approximates the greedy policy $\arg\max_a Q(s, a)$ restricted to the support of the learned flow policy. The intuition is that even if the policy assigns probability to suboptimal actions, rejection sampling filters these out at evaluation time. With a random policy, rejection sampling would produce uniformly random actions regardless of N; the benefit requires a policy that places at least some mass on high value actions.

5.2.4 DSRL

DSRL ($\sigma_z = 1.0$ for cube double, $\sigma_z = 0.8$ for antsoccer) achieves 60% on cube double and 60% on antsoccer arena after 500k steps. Operating in latent noise space offers three benefits. First, the action space behavioral cloning policy is fixed, so the RL updates cannot destabilize it and the expensive flow model does not need to be optimized by the online RL objective. Second, exploration is easier because the learned policy only searches for useful latent noise inputs to a pretrained decoder, rather than directly exploring the full action space. Third, the method only needs access to the BC model as a decoder from latent noise to action; the RL policy and critic can be trained in latent space without requiring the BC model itself to remain adaptable. The noise space critic Q^z then provides value estimates in a space where the mapping to actions is already well structured, making optimization easier.

5.2.5 QSM

QSM ($\eta = 30$, $\alpha = 30$) achieves 64% on cube double after 300k steps and 48% on antsoccer arena after 400k steps. The Q score matching objective is justified because learning $\pi(a|s) \sim e^{\eta Q(s,a)}$ implies that the

score $\nabla_a \log \pi(a|s) = \eta \nabla_a Q(s,a)$. Since diffusion/flow models learn score functions, aligning the model's predicted noise with the Q gradient steers the denoising trajectory toward high value actions. The DDPM loss (weighted by α) ensures the flow model still produces valid actions, preventing mode collapse toward a single high Q direction.

5.3 QAM Attempt

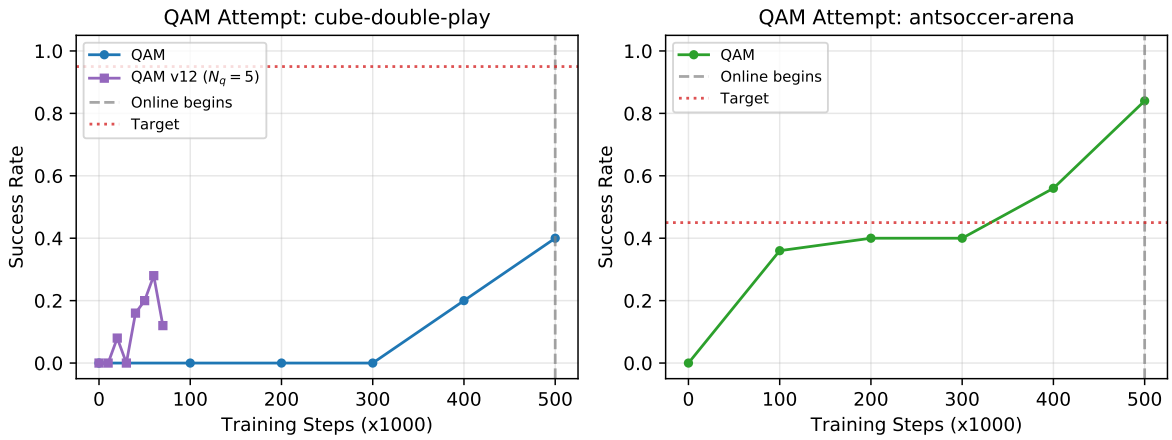


Figure 5: Archived QAM runs from our custom method exploration. QAM reaches the antsoccer target on this seed but remains far below the cube double threshold.

Figure 5 shows the archived QAM run used during our custom method exploration. On antsoccer arena, base QAM with $\eta = 3.0$, $\rho = 0.0$, $K = 2$, $T = 5$, and adjoint updates every fourth step reaches 84% success at the 500k offline checkpoint, exceeding the 45% target for this environment. On cube double, the archived single step QAM run uses $\eta = 0.3$, $\rho = 0.5$, $K = 2$. It reaches only 40% success at 500k, far below the 95% target. A later cube double QAM run with $\eta = 1.0$, $\rho = 0.5$, and $N_q = 5$ peaked at 28% success after 60k logged steps and ended at 12% after 70k logged steps. Later chunked runs gave stronger early learning, which supported our conclusion that action chunking is a necessary integration detail for cube double, but they did not become the final submitted method because PAWFQL was more stable and performant.

5.4 Comparison of Method to SoTA

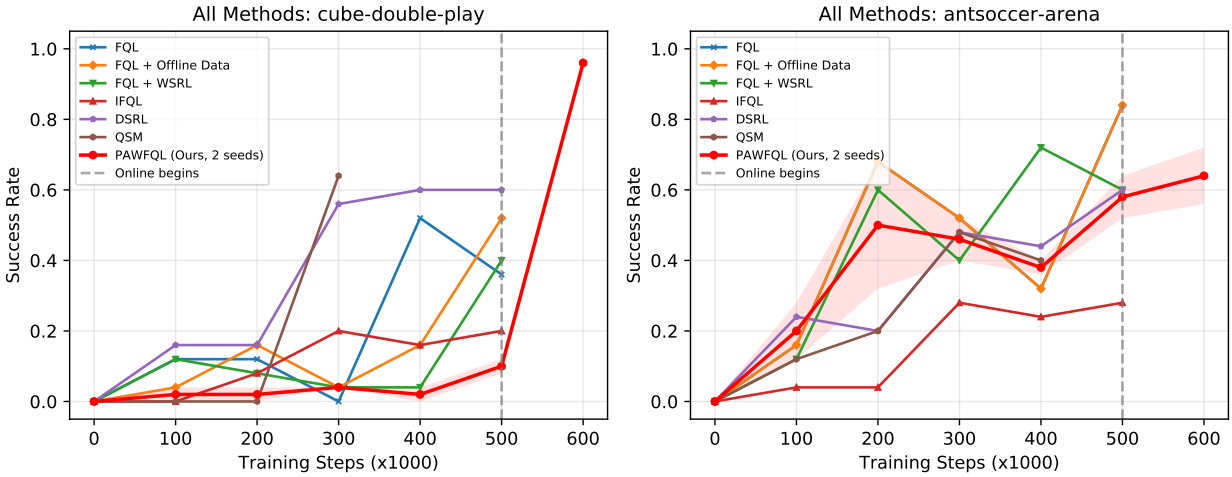


Figure 6: Comparison of all methods on cube-double-play and antsoccer-arena. PAWFQL gives the strongest cube double result and exceeds the required threshold on antsoccer arena.

Figure 6 shows the full comparison, including the naive FQL baseline on the two target environments. On cube-double-play, PAWFQL achieves 96% average success rate after 600k total steps (500k offline + 100k online), compared with 64% for the next best method, QSM. On antsoccer arena, PAWFQL reaches 64% averaged over two seeds, above the 45% target, while the strongest single seed FQL and FQL with offline data runs reach 84%.

Method	cube double success	antsoccer success
FQL	52%	84%
FQL + Offline Data	52%	84%
FQL + WSRL	40%	72%
IFQL	20%	28%
DSRL	60%	60%
QSM	64%	48%
PAWFQL	96%	64%

Table 1: Best logged evaluation success rates for the baselines in Figure 6. PAWFQL reports the two seed average at the final evaluation checkpoint.

PAWFQL's advantage is clearest on cube double, where coordinated manipulation makes policy selection and value overestimation especially important. First, advantage weighted flow matching focuses the generative model on high value transitions during online finetuning, effectively performing importance weighted behavioral cloning that improves as the critic becomes more accurate. Second, pessimistic action selection via best of N with min Q avoids the overestimation that can destabilize online finetuning, particularly in the early online phase when the critic has not yet adapted to the new data distribution.

The hyperparameter settings differ between tasks. For cube double, a high distillation coefficient ($\alpha = 1000$) with balanced replay (50% offline, 50% online) [12] prevents catastrophic forgetting while still allowing online improvement. For antsoccer arena, a lower $\alpha = 10$ with a 100k warm start period allows more aggressive online adaptation. This task specific tuning is consistent with the broader observation across all methods that the regularization strength must be adapted to the difficulty and data quality of each environment.

6 Discussion and Limitations

We have implemented and evaluated seven offline to online RL methods on OG-Bench tasks. The naive baselines (SAC+BC and FQL) demonstrate that simply continuing training online can work on simple tasks but faces challenges on harder ones. The improved baselines each address a specific aspect of the offline to online transition: data retention smooths distribution shift, WSRL calibrates the replay buffer to the deployment distribution, IFQL improves test time action quality, DSRL provides stable optimization via latent space RL, and QSM aligns the generative model with Q value structure.

PAWFQL synthesizes insights from several of these methods. Like AWAC, it uses advantage weighting to focus learning on high value experience. Like DSRL and IFQL, it employs pessimistic value estimates for stable optimization. The key contribution is applying these ideas within the FQL framework, where the flow matching loss provides a natural target for importance weighting.

A limitation of our approach is the sensitivity to the activation step for pessimistic mechanisms and the task specific replay strategy. Future work could explore automatically scheduling the transition from uniform to advantage weighted training, or learning the replay ratio online based on value function uncertainty.

7 Contributions

Jai Sharma contributed to method implementation, experiment execution, figure generation, analysis, and report writing. He was mainly in charge of SAC+BC, FQL, offline data retention, WSRL, IFQL, DSRL, QSM, and QAM.

Socrates Osorio contributed to method implementation, experiment execution, analysis, and report writing. He was mainly in charge of the main PAWFQL method.

References

[1] S. Fujimoto and S. S. Gu. A minimalist approach to offline reinforcement learning. In *NeurIPS*, 2021.

[2] S. Park, K. Frans, B. Eysenbach, and S. Levine. OGBench: Benchmarking offline goal-conditioned RL. In *ICLR*, 2025.

[3] S. Park, Q. Li, and S. Levine. Flow Q-learning. In *ICML*, 2025.

[4] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit Q-learning. In *ICLR*, 2022.

[5] Q. Li and S. Levine. Q-learning with adjoint matching. arXiv preprint arXiv:2601.14234, 2026.

[6] Q. Li, Z. Zhou, and S. Levine. Reinforcement learning with action chunking. arXiv preprint arXiv:2507.07969, 2025.

[7] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. In *CoRL*, 2025.

[8] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma. Learning a diffusion model policy from rewards via Q-score matching. In *ICML*, 2024.

[9] A. Nair, A. Gupta, M. Dalal, and S. Levine. AWAC: Accelerating online reinforcement learning with offline datasets. arXiv preprint arXiv:2006.09359, 2020.

[10] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. arXiv preprint arXiv:2304.10573, 2023.

[11] Z. Zhou, A. Peng, Q. Li, S. Levine, and A. Kumar. Efficient online reinforcement learning fine-tuning need not retain offline data. In *ICLR*, 2025.

[12] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. In *ICML*, 2023.